ISA 401: Business Intelligence & Data Visualization

06: SQLite and Your First Deployed App

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Open the project

GitHub Desktop → `business_intelligence` → **Fetch origin**. Then double-click **business_intelligence.Rproj**.
Top right corner of RStudio must say `business_intelligence` before you type anything.

3 Start today's markdown

File → New File → R Markdown..., title *Class 06*, **OK**. Delete everything below the setup chunk, then save as **markdowns/class06.Rmd**.
Same as Wednesday. Your `class05.Rmd` stays as it is.

4 Get ready for Kahoot

Open **kahoot.it** in a browser tab. Two of the questions ask you to read a file in R, so **run the two lines on the next slide** in your markdown now.
The file is Section A's Find Your Seat run from Wednesday: one row per seat per level.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Review on Kahoot: Classes 03 to 05

How to play

1. Go to **kahoot.it** on your phone or in a browser tab.
2. Enter the **game PIN** shown on the projector.
3. Use your **first and last name** so the bonus can find you.
4. **20 seconds** per question; the two data questions get **30**. Faster correct answers score more, and every question is worth double.

What is in it

8 questions on everything so far:

- 6 brackets, Excel, paths, JSON, and the Heist
- 2 read straight off `glimpse()` of Wednesday's run, loaded below

The winner 🏆 receives a **0.15 bonus** on Assignment 05.

Copy

```
run = readr::read_csv(
  "https://isa401-find-your-seat.vercel.app/api/export?section=A&run=13")
dplyr::glimpse(run)
```

> The competition has **no effect on your grade** beyond the bonus. It tells you which ideas need another look, and it tells me how to pace today.

3 / 30

Learning Objectives for Today's Class

- **Import** a table from a **SQLite database file** with `DBI`, then **ask the database a question in SQL** with `dbGetQuery()`.
- **Build** a `querychat` app on the Job Scout data and **deploy** it to **Render** from your GitHub repository.
- **Read the SQL** an app writes for you, and **explain** what a **data dictionary** adds to an app that turns questions into SQL.

PART 1	PART 2	PART 3	PART 4
A database file	**Your first app**	**Deploy**	**Read the SQL**
scout.db, one SQL query	A dozen lines of R	Render, from GitHub	Try it, then Wednesday's dplyr

Part 1: A Database File

scout.db: 1,891 Postings in One File

```
con = DBI::dbConnect(RSQLite::SQLite(),
                     "../data/scout.db")
DBI::dbListTables(con)
postings = DBI::dbReadTable(con, "scout_postings")
DBI::dbDisconnect(con)
dplyr::glimpse(postings[, c("title", "company",
  "location_state", "remote", "skills_json")])
```

```
[1] "scout_postings" "scout_runs"
```

```
Rows: 1,891
Columns: 5
$ title          <chr> "Data Analyst - Remote", "B…
$ company        <chr> "UnitedHealth Group", "Anza…
$ location_state <chr> "GA", "NC", "TX", "CA", "TX…
$ remote         <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, …
$ skills_json    <chr> "[{\"skillId\":\"sql\",\"im…
```

The fourth format: a database in a file

SQLite packs whole tables into one `.db` file. This one is the database behind ChatISA's Job Scout: two tables, `scout_postings` (one row per ad) and `scout_runs` (one row per harvest).

Three steps, always: connect, read the table you want into a data frame, disconnect. From here on it is a data frame like any other.

Download `scout.db` from the **Week 0 module** into `data/`. You already ran these three steps on `midwest_airbnb.db` for Assignment 04; today the file is the Job Scout database.

JSON Inside a Cell

```r
first_cell = postings$skills_json[1]
cat(strtrim(first_cell, 60))    # base R: first 60 characters
```

```
[{"skillId":"sql","importance":"required"},{"skillId":"excel
```

```r
skills = jsonlite::fromJSON(first_cell)
skills
```

```
             skillId importance
1                sql   required
2              excel   required
3      data_analysis   required
4  dashboard_design   required
5      communication   required
6 presentation_skills   required
```

A list hiding in a data frame

Each cell of `skills_json` is a JSON **array of records**, exactly the shape from last week's list slides. `fromJSON()` on one cell returns a small data frame: one row per skill, tagged required or preferred.

A table with 1,891 of these is **semi-structured** in a structured wrapper. Flattening all of them into one long table is a Week 7 job (tidy data).

This is why `[[ ]]` and `$` mattered: `postings$skills_json[1]` is **vector, then element**; `fromJSON()` then turns text into structure.

Ask the Database a Question in SQL

```
con = DBI::dbConnect(RSQLite::SQLite(),
                     "../data/scout.db")
top_remote = DBI::dbGetQuery(con, "
  SELECT company, COUNT(*) AS n
  FROM scout_postings
  WHERE remote = 1
  GROUP BY company
  ORDER BY n DESC
  LIMIT 10")
DBI::dbDisconnect(con)
top_remote
```

	company	n
1	The Home Depot	5
2	Gainwell Technologies LLC	5
3	Jerry.ai	4
4	Burjline Builders	4
5	TEKsystems	3
6	OhioHealth	3
7	Centene Corporation	3
8	Zest AI	2
9	UnitedHealth Group	2
10	TRILLIUM HEALTH RESOURCES	2

Which companies posted the most remote jobs?

`SELECT`	which columns to return; `COUNT(*)` counts rows and `AS n` names the count
`FROM`	which table
`WHERE`	keep only the rows that match
`GROUP BY`	one output row per company
`ORDER BY`	sort, `DESC` for largest first
`LIMIT`	stop after ten rows

The database did the work: 1,891 rows stayed in the file and ten came back. `dbReadTable()` is `SELECT * FROM scout_postings` with no filter. This is the language the app in Part 2 writes for you, and on Wednesday you will ask the same question in dplyr.

In the job market: SQL appears in 602 of the 1,891 Job Scout posting descriptions (31.8%); Python 522 (27.6%), Power BI 318 (16.8%), Tableau 251 (13.3%). Source: `data/scout.db`, harvest through 2026-08-23.

Part 2: Your First App

What querychat Does

CHAT, IN THE APP

OPENAI MODEL RECEIVES

your question lands here

- **company** (TEXT)
- **title** (TEXT)
- **remote** (INTEGER) Range: 0 to 1
- **location_state** (TEXT)
- **category** (TEXT) Categorical values: 'federal', 'fulltime', 'internship'

rows received: 0

THE MODEL WROTE one possible query

RENDER SERVER · SQLITE · SCOUT.DB · SCOUT_POSTINGS, 1,891 ROWS · READ-ONLY: SELECT ONLY



the rows stay here

eight of the 1,891 rows, chosen to show every clause

COMPANY	TITLE	REMOTE	N
The Home Depot	Data Scientist - People Analytics	1	
CVS Health	Data Engineer	1	
OhioHealth	Requirement Business Analyst	0	
The Home Depot	Cybersecurity Analyst | PCI (Remote)	1	
Talkspace	Data Analyst	1	
The Home Depot	Business Analyst	0	
CVS Health	Data Scientist (Decision Sciences Team)	1	
The Home Depot	Data Scientist, Pricing (Optimization & Deep	1	

Watch eight rows. These are eight of the 1,891 rows in `scout_postings`, sitting on the server; nothing has happened yet.

Back | Step | Play | Reset

0.5x | 1x | 2x

0 / 7

What it costs. Step 3 is the only paid part: each question is one call to OpenAI on a class key I hand out today. Everything else is free: `querychat` is open source, and Render's free tier hosts a small app.

10 / 30

A Dozen Lines of app.R

```
# apps/job_scout_chat/app.R
library(querychat)

con = DBI::dbConnect(RSQLite::SQLite(), "data/scout.db")

client = ellmer::chat_openai(
  model  = "gpt-5.6-luna",
  params = ellmer::params(reasoning_effort = "none")
)

qc = querychat(
  con, "scout_postings",
  client   = client,
  tools    = c("filter", "query", "visualize"),
  greeting = "Ask me about the 1,891 job postings
              ChatISA Job Scout collected."
)

qc$app_obj()
```

The path is `"data/scout.db"`, not `"../data/"`: an app runs from **its own folder**, so the app gets its own copy of the data.

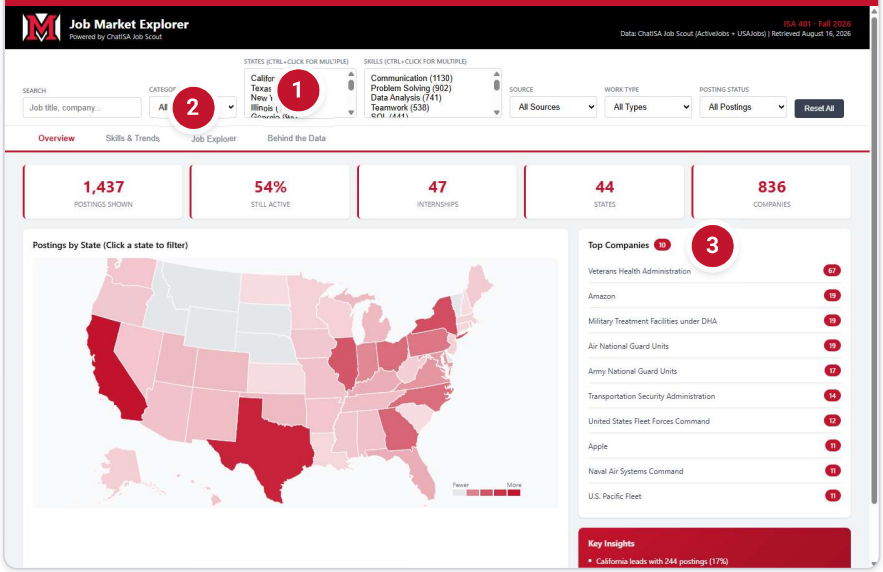
Run it on this computer

1. Unzip `job_scout_chat.zip` from Canvas into your repo as `apps/job_scout_chat/`. It holds `app.R`, the `Dockerfile`, `README.md`, and `data/scout.db`.
2. Open `app.R` from that folder. It is the code on the left; type it yourself if you prefer.
3. In the Console: `install.packages("querychat", dependencies = TRUE)` (pulls in `DT` and `ggsql`, which the app needs but querychat only suggests), then `usethis::edit_r_environ("project")`, add the line `OPENAI_API_KEY=` followed by the **class key** (Canvas), save. Then `usethis::use_git_ignore(".Renviron")`, and **Session → Restart R**.
4. Click **Run App** at the top of `app.R`.

The key never goes in a file you commit. `.Renviron` from step 3 sits next to your `.Rproj`, so it follows your M: drive to any lab PC; the `use_git_ignore()` line keeps it out of every commit.

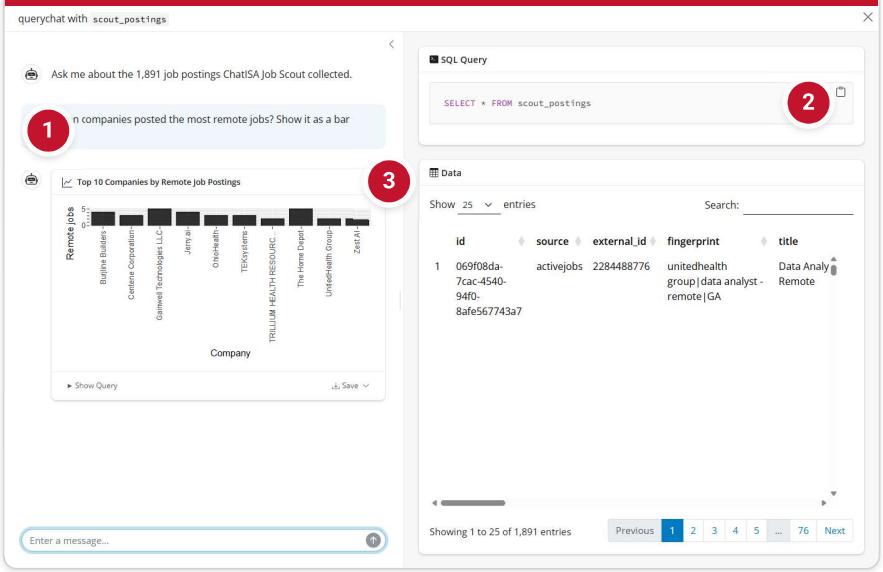
Your First Dashboard

Class 01: the Job Market Explorer widgets someone built in advance



1. The **States** and **Skills** list boxes: every filter a designer anticipated
2. The **Category** and **Work type** dropdowns
3. The **Top Companies** panel, one fixed chart

Today: your querychat app the same 1,891 postings, one sentence at a time



→ one sentence in the chat; the model turns it into `WHERE location_state = 'OH'`

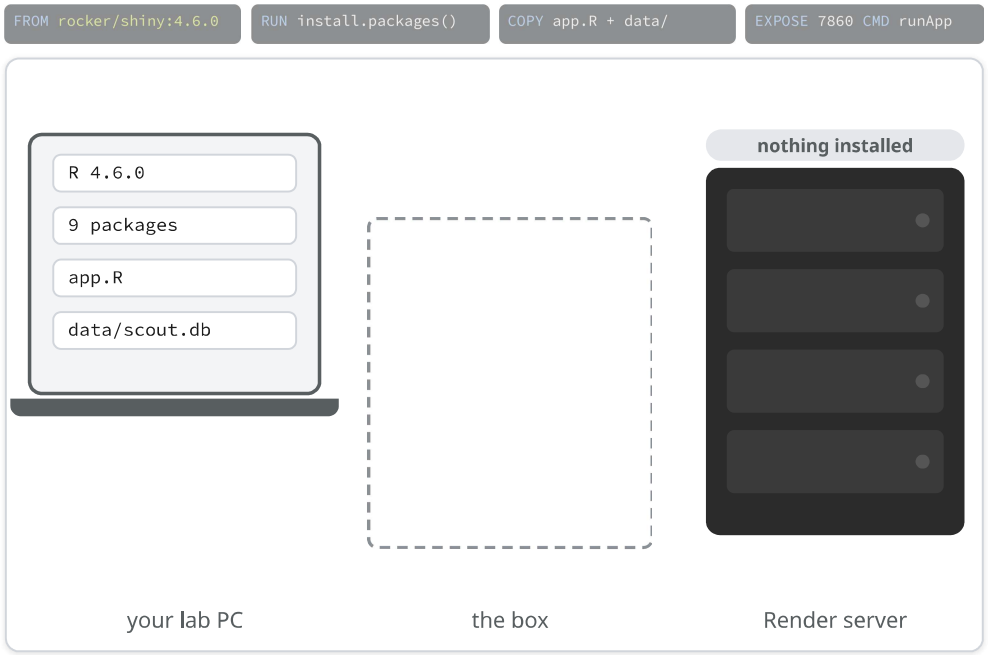
→ the SQL card: `WHERE category = 'internship'`, or any condition you can say in English, shown so you can check it

→ a chart in the chat when you ask for one: `GROUP BY company ORDER BY n DESC` plus a VISUALISE clause

This is your first dashboard. A dashboard is filters plus views of a table. The Explorer's designer had to guess every filter and chart in advance; your app builds each one from a sentence, and shows you the SQL it used. What it cannot do: anything the schema does not contain.

Part 3: Deploy

Why a Dockerfile? Ship the Computer



Before the recipe. Your app runs on this PC only because R, nine packages, app.R, and scout.db all happen to be here. The server on the right has none of them.

Back | Step | Play | Reset | 0.5x | 1x | 2x | 0 / 6

Why it matters to you

In the **1,891** Job Scout postings, the description mentions:

- **351** deploy or deployment
- **249** AWS, Azure, or Google Cloud
- **31** Docker
- **31** Kubernetes

Deploying is a common ask. Docker itself is named only in machine-learning and data-engineering roles: recognize it, do not fear it.

Industry-wide

71.1% of the 24,473 developers who answered the Cloud development question in the Stack Overflow 2025 Developer Survey use Docker. A Dockerfile is a recipe for a computer; Hugging Face reads the recipe, so nobody installs anything by hand.

Three words from job postings: **recipe** (Dockerfile) → **box** (image) → **running box** (container).

Sources: `data/scout.db` (ChatISA Job Scout harvest through 2026-08-23; counts are case-insensitive matches in the posting text); Stack Overflow 2025 Developer Survey, Technology, Cloud development; Docker: What is a container.

Deploy to Render

1 Account

Sign up at **render.com** with **GitHub**, using the account that owns `business_intelligence`. The Hobby plan is free and asks for no card. Skip the onboarding questions.

2 Connect the repository

New → Web Service → Git Provider → GitHub. Install Render's GitHub app on `business_intelligence` (*Only select repositories*), then click the repository to connect it. Your app folder is already in it from the ritual.

3 Configure

Name `job-scout-chat`. Language **Docker** (detected from the Dockerfile). Branch `main`. **Root Directory** `apps/job_scout_chat`. Instance type **Free**, $0 (the default is paid; change it). Environment variable `OPENAI_API_KEY` = the **class key**. **Deploy Web Service**.

4 Watch it build

Logs run the Dockerfile line by line; a few minutes the first time. Status **Live**, address `https://job-scout-chat-xxxx.onrender.com`. A free app sleeps after 15 idle minutes and wakes in about a minute; a 502 while waking is normal.

What the Dockerfile is. A recipe that builds a fresh Linux computer with the same R version as this lab, installs the packages from a dated snapshot, copies your app in, and starts it. You do not run Docker; Render does. Same variable name as the line in your `.Renviron`: `app.R` reads `OPENAI_API_KEY` from wherever it runs, so the key is never in a file you commit. The recipe is read line by line in Appendix A1.

Where Your App Will Live

$0
a month for a free instance: 0.1 CPU, 512 MB of memory

15 min
idle before a free app sleeps; about a minute to wake

500
build minutes a month, and 750 hours of free instance time

What changed this month

Sep 2026	Hugging Face makes Docker Spaces a paid feature: "Gradio and Docker Spaces run on compute and require a paid plan to create."
Sep 17	The reference app moves to Render in one afternoon: the same Dockerfile, one line changed (the port), a new URL.

Render is a hosting company: you hand it a repository with a recipe, it builds the computer, runs it, and gives it an address. The app you deploy today is one of its free instances.

The business story

Render sells compute by the month: a small paid instance is $7, a Pro workspace $25 plus compute. The free tier is the top of that funnel: it costs the company real machines and earns nothing until a hobby project becomes a business.

Free tiers change when a company's economics change, which is what Hugging Face did this month. A Dockerfile is what makes your app portable, so a platform change costs an afternoon, not a rewrite.

Question for the room. Why would a company give away compute? What does it get from a student who deploys one app and never pays?

Sources: Render pricing and Render free instances (limits as shown in a Hobby workspace on Sep 17, 2026); Hugging Face Spaces overview (the paid-plan sentence, read Sep 16, 2026).

Part 4: Read the SQL

Try It

Four questions for your app

1. Which ten companies posted the most remote jobs?
2. Show the remote postings in Ohio.
3. How many companies require R and data visualization?
4. Question 1 again, but: show it as a bar chart.

Then read the SQL, not just the table

Did question 1 come back with your Part 1 query: `WHERE remote = 1`, `GROUP BY company`, the same ten names?

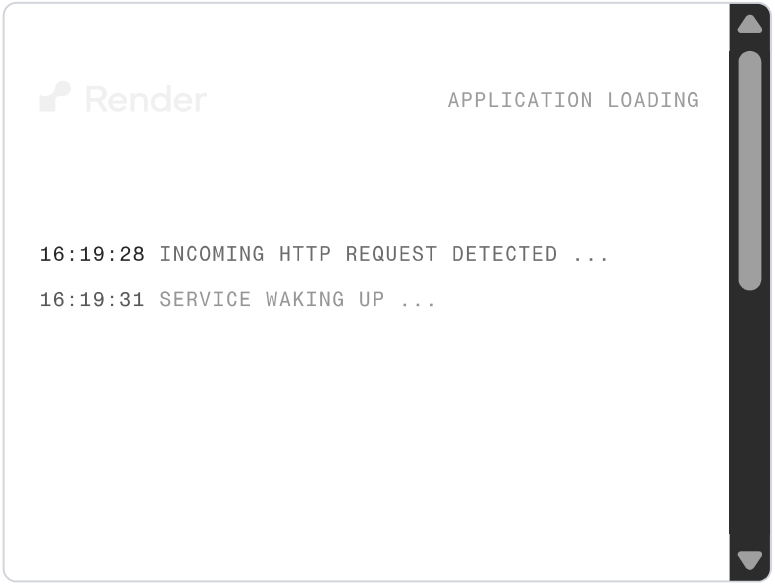
Did question 2 filter on `location_state = 'OH'` or on the word *Ohio* somewhere?

Did question 3 search `skills_json` or `description`, and did it count companies or postings? Each is defensible; only one is what you meant.

Question 4: the chart appears in the chat, the table stays. Open the chart's query: same SQL as question 1?

Ask the same question twice. Same SQL?

Still building? The reference app, live: **job-scout-chat.onrender.com** (source: fmegahed/job_scout_chat). Same data and the same dozen lines, plus a bslib layout (Appendix A3 and a little more). Click **Explorer**, then the chat toggle.



The model does not know what a column means. It sees names, types, and ranges: nothing tells it that `location_state` holds two-letter codes. Telling it, in a **data dictionary**, is Appendix A2 and most of Assignment 05.

Three Ways to Ask the Same Question

Which companies posted the most remote jobs? You wrote it in SQL in Part 1. The app wrote the same SQL from your English in Part 4. On Wednesday, you write it in R.

```
# Today: SQL, by you (Part 1) or by the app (Part 4)
DBI::dbGetQuery(con, "
  SELECT company, COUNT(*) AS n
  FROM scout_postings
  WHERE remote = 1
  GROUP BY company
  ORDER BY n DESC
  LIMIT 10")
```

```
# Wednesday: dplyr, on the data frame from dbReadTable()
postings |>
  dplyr::filter(remote == 1) |>
  dplyr::count(company, sort = TRUE) |>
  head(10)
```

Same verbs, R spelling

`WHERE`	`filter()`
`GROUP BY` + `COUNT(*)` + `ORDER BY`	`count(sort = TRUE)`
`LIMIT`	`head()`

The `|>` between the lines reads as **then**: take `postings`, then keep the remote rows, then count per company, then the first ten.

Nothing to type today. Wednesday is **filter, select, mutate, arrange, group_by, summarize**, and joins: the friendlier spelling of every bracket you have written since Class 04.

Our Class Git + GitHub Ritual

1 Knit

Click **Knit**. `class06.html` re-runs every chunk top to bottom and saves the page next to your markdown.

If the knit fails, a chunk depends on something you only ran in the Console. Fix the markdown, not the Console.

→

2 Commit

GitHub Desktop lists today's work: `markdowns/class06.Rmd`, `class06.html`, `data/scout.db`, `apps/job_scout_chat/`, and `.gitignore` with its new `.Renviron` line.

If `.Renviron` itself is in the list, stop: uncheck it and redo the `use_git_ignore()` step. Summary: `Add class 06 markdown, scout.db, and the job scout app.`

→

3 Push and verify

Push origin, then check the repo on GitHub: `markdowns/` and `data/` both there, readable from any device.

That is code, data, and output, together.

If it is not pushed, it is not done. Your repo now carries code, data, and output that anyone, including future you, can re-run.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Import** a **SQLite table** (`dbConnect()`, `dbReadTable()`, `dbDisconnect()`) and **query** it with `dbGetQuery()`: `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `ORDER BY`, `LIMIT`.
- **Build and deploy** a `querychat` app: a dozen lines of R, a Dockerfile (a recipe that builds the same computer anywhere), one secret, one public address.
- **Read the SQL** an app writes before trusting its table, and say what a **data dictionary** would have fixed.

Skills practiced: `glimpse()` on every import, JSON inside a cell, one question asked three ways.

Reading for Assignment 05: A1 the Dockerfile, A2 the data dictionary, A3 polishing with bslib, A4 deploying with git. **Next class (Wed, Sep 16):** dplyr, the same questions in R verbs.

Before You Go

Join at **https://www.menti.com/al37nbpvnzpg**

Before you go: one thing from today you would like clarified.

Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

Assignment 05: Deploy Your Midwest Airbnb Chat

Today's recipe on the Midwest Airbnb listings: same Dockerfile, new data, a full data dictionary, extra instructions, a theme and an About section of your own (the A3 code is enough; no tabs or value boxes required). Submit the folder's GitHub address; the live Render URL goes in its README. Appendices A1 to A4 are the reading. Due date on Canvas.

Before you leave: knit, then commit and push `markdowns/class06.Rmd`, `class06.html`, `data/scout.db`, and `apps/job_scout_chat/`. If it is not pushed, it is not done.

Ungraded Practice Questions

1. Change the Part 1 query to count postings per `location_state`, largest first, top five. Which state is first, and with how many?

2. Read the second table, `scout_runs`, from `scout.db` and `glimpse()` it. How many harvests are recorded?

3. Parse `skills_json` for row 50. How many skills are required?

4. Ask your app the question from 1. Does its SQL have the same `GROUP BY` and `ORDER BY` as yours?

5. Ask your app for the remote postings in Ohio, then read the `WHERE`. What would a data dictionary have told the model?

▶ **Answers**

Appendices

Not covered live. The reading for Assignment 05.

A1: The Dockerfile, Line by Line

```dockerfile
FROM rocker/shiny:4.6.0

RUN apt-get update && apt-get install -y --no-install-recommends \
    libcurl4-openssl-dev libssl-dev libxml2-dev libuv1t64 \
    && rm -rf /var/lib/apt/lists/*

RUN echo 'options(repos = c(CRAN =
  "https://p3m.dev/cran/__linux__/noble/2026-09-01"))' \
    >> /usr/local/lib/R/etc/Rprofile.site

RUN R -q -e "install.packages(c('shiny', 'bslib', 'DBI', 'RSQLite',
  'ellmer', 'shinychat', 'querychat', 'DT', 'ggsql'))"

RUN R -q -e "stopifnot(requireNamespace('querychat'),
  requireNamespace('RSQLite'), requireNamespace('DT'),
  requireNamespace('ggsql'), requireNamespace('fs'))"

WORKDIR /app
COPY app.R app.R
COPY data/ data/

# Render sets PORT at runtime; local Docker uses 7860
EXPOSE 7860
CMD ["R", "-q", "-e", "shiny::runApp('/app', host = '0.0.0.0',
  port = as.integer(Sys.getenv('PORT', '7860')))"]
```

`FROM`: start from a ready-made Linux image with **R 4.6.0** and Shiny Server, the same R as this lab. Change this number and the packages below may no longer match.

`apt-get`: system libraries that `ellmer` (web requests) and `RSQLite` need, plus `libuv1t64`, which this rocker image forgot to ship and Shiny's file helpers need. Found by building the image, not by reading about it.

`options(repos = ...)`: install every package from **one dated snapshot** of CRAN. A build in December installs the same versions as today. Last year's version of this file pinned versions from a repository that only keeps the latest build, and it broke the week a package updated.

`install.packages(...)` lists `DT` and `ggsql` too: querychat only suggests them, but the app cannot draw its table without DT or its charts without ggsql.

`stopifnot(...)`: fail the build now, with a readable log, rather than at the first visitor.

`COPY`, `EXPOSE`, `CMD`: copy your app and data in, open a door, and start Shiny on the port the host hands the container (Render sets `PORT`; 7860 is the fallback for a laptop).

Sources: Rocker images; Posit Package Manager snapshots; Render: Docker deploys.

A2: Teaching the App About Your Data

```
qc = querychat(
  con, "scout_postings",
  client             = client,   # same ellmer client as ap
  greeting           = "Ask me about the postings.",
  data_description   = "data/data_desc.md",
  extra_instructions = "data/extra_instructions.md"
)
```

data_desc.md: the data dictionary

What the table is, where it came from, and **one row per column**: name, meaning, units, allowed values. Example: `remote` is 1 for remote-eligible, 0 otherwise; `location_state` is a two-letter code; `category` is one of `fulltime`, `federal`, `internship`.

extra_instructions.md: house rules

Short rules for the model: *"Match states by their two-letter code." "Skills live in `skills_json`; search it, not `description`." "Return at most 50 rows unless asked."*

Before and after

Question: *Show the remote postings in Ohio.*

Without a dictionary: `WHERE location_state LIKE '%Ohio%' AND remote = 1`. Zero rows: the column holds `OH`.

With it: `WHERE location_state = 'OH' AND remote = 1`. The rows you meant.

Both files go in the model's **system prompt**, once, at startup. Writing them is the same work as documenting a dataset for a colleague, which is why the data dictionary is a deliverable in Assignment 05 and in every project this semester.

Starter files with the first six columns documented are in the `data/` folder of `job_scout_chat.zip` on Canvas.

Source: querychat: Provide Context, the `data_description` and `extra_instructions` arguments.

A3: Polishing With bslib

```
library(shiny)
library(bslib)

ui = page_sidebar(
  title   = "Job Scout Chat",
  theme   = bs_theme(primary = "#C3142D",
                     base_font = font_google("Lato")),
  sidebar = qc$sidebar(width = 350),
  card(card_header(textOutput("title")),
       DT::DTOutput("table")),
  accordion(open = FALSE,
    accordion_panel("SQL", verbatimTextOutput("sql")),
    accordion_panel("About", "Job Scout postings; built by <your name>"))
)

server = function(input, output, session) {
  vals = qc$server()
  output$title = renderText(vals$title() %||% "All postings")
  output$table = DT::renderDT(vals$df(),
                              options = list(pageLength = 10))
  output$sql   = renderText(vals$sql() %||%
                              "SELECT * FROM scout_postings")
}

shinyApp(ui, server)
```

Replace `qc$app_obj()` with your own layout

`qc$sidebar()` is the chat; **`qc$server()`** returns three reactives: `df()`, `sql()`, `title()`. Everything else is ordinary Shiny.

`bs_theme()` sets colors and fonts in one place; **`card()`** and **`accordion()`** are the building blocks.

This is all Assignment 05 asks for: a theme and title of your own, an About section naming the data and you, and the SQL panel visible. The code on the left, with your names in it, meets all three.

Optional, not graded: the reference app job-scout-chat.onrender.com goes further, with `page_navbar()` tabs, `value_box()` counts, and icons on top of this layout; the source is at github.com/fmegahed/job_scout_chat. Read it for ideas, not as the target. If you borrow icons, add `bsicons` to the Dockerfile's `install.packages()` line.

Sources: bslib: Get Started; querychat: Build an App.

A4: How Render Builds From GitHub

Every push is a deploy

1. Render's GitHub app watches the branch you connected. A push that touches the **root directory** (`apps/job_scout_chat`) starts a build; a push elsewhere in the repository does not.

2. The build runs your Dockerfile on Render's machine: pull `rocker/shiny:4.6.0`, install the packages from the dated snapshot, copy the app in. Each build costs a few of the 500 build minutes a free workspace gets per month.

3. The new container replaces the old one and the address stays the same. The **Events** tab lists every deploy with its commit; **Logs** shows the build and the running app.

4. Assignment 05 asks for at least three commits to the app folder; each is its own deploy, and the folder's history on GitHub is what gets graded.

The .gitattributes file in the zip

`Dockerfile text eol=lf` (and `*.R`, `*.md`): a Windows editor can turn line endings into CRLF, and a carriage return after a trailing backslash breaks the Dockerfile on Linux. This line keeps them LF on every machine.

No Git LFS line any more: GitHub takes files up to 100 MB as ordinary files, and both databases are far below that.

If you committed the class app before Sep 17 with the old `.gitattributes`, `scout.db` is stored through Git LFS and Render's build may see only a pointer file. Fix: delete the `*.db filter=lfs` line, remove `scout.db` from the folder, commit, put it back, commit again.

Sources: Render: deploys and auto-deploys; Render: root directories; Render: free instances; GitHub: file size limits.